

Glassbox for IT

What it reads, what it writes, what it runs, what it sends. Written for the person who has to approve it, checked against the source of glassbox-trace 0.9.1. If anything here stops being true, that is a bug — open an issue.

Glassbox is a viewer and a checker for the transcript files Claude Code already writes to every developer's machine. It has no server, no account, no telemetry and no runtime dependencies. Everything below runs as the user, on the user's machine.

1. The three ways it can be on a machine

	what lands on the machine	how it gets there	how it runs
The HTML file	one file, <code>glassbox.html</code> (595 KB; fonts, code and demo inside)	download from GitHub Releases / npm, or copy from an internal share	double-click; runs in the browser from <code>file://</code>
The CLI	npm package <code>glassbox-trace</code> : 14 files, ~325 KB packed, zero dependencies (<code>package.json</code> has no <code>dependencies</code> field); published with npm provenance from the GitHub Actions workflow in this repo	<code>npx glassbox-trace</code> (fetches on each run unless cached) or <code>npm i -g glassbox-trace@0.9.1</code> (fetches once, pinned)	<code>node</code> ≥ 18
The Claude Code plugin	a clone of this repository under Claude Code's plugin directory	<code>/plugin marketplace add aldohushi1-stack/glassbox</code> then <code>/plugin install glassbox@glassbox-trace</code>	<code>node</code> running the bundled copy; no npx, no network after install

For a managed fleet the simplest shape is: the HTML file on an internal share (nothing to install, nothing to update automatically) plus, where the CLI is wanted, a global install of a pinned version.

2. Network

The viewer makes no network requests. Fonts, scripts and the demo session are inside the file. This is enforced by the test suite: `test/offline.test.mjs` fails the build if the file references anything that is not a `data:` URL, and the browser test logs every request the page makes and fails if any is not `file:`, `data:` or `blob:`. (Versions up to 0.6.1 loaded one stylesheet from `fonts.googleapis.com`; that is gone.)

The CLI makes no network requests. `list`, `open`, `check`, `compare`, `reveal`, `collect`, `clean`, `fence`, `adhere`, `hook` and `watch` open no sockets to anything outside the machine.

`glassbox watch` starts an HTTP server bound to `127.0.0.1` on a random free port (or `--port N`) for the lifetime of the command, to stream the transcript to the browser tab it opens. It is not reachable from other machines and stops with `Ctrl+C`.

The only network activity is installation: npm fetching the package (`npx` does this on each run if the package is not cached; a global install does it once), or Claude Code's plugin marketplace cloning this repository.

3. Files it reads

- `~/.claude/projects/<project>/<session-id>.jsonl` and the `<session-id>/subagents/**` beside it — Claude Code's own transcripts. `GLASSBOX_HOME` or `--home` points at a different `.claude` directory. `list` reads directory entries and file sizes; `list --grep` reads every transcript's full text

to search it; `open`, `check`, `compare`, `watch` read the session(s) named; the loop guard (`$6`, only with `--guard`) reads the last 4 MB of the current session's transcript before each tool call.

- A `.jsonl` file or folder the user names on the command line or drops onto the page.
- `~/.claude/settings.json` — only by `hook install` / `hook uninstall`, and by the plugin hook to see whether a CLI hook is already installed.
- `--rates FILE` / `GLASSBOX_RATES` (a rate card), `--legend FILE` (a legend), `--key FILE` (a fence key), when given.
- `glassbox fence` with no target also reads `~/.claude/history.jsonl`, `paste-cache/`, `file-history/`, `debug/` and `shell-snapshots/` (`$6b`).
- In the browser, **Open folder...** uses the File System Access API (Chrome/Edge): the user picks a folder, the browser asks permission, and the page reads `.jsonl` files under it. The permission is per site and per folder, revocable in the browser.

These transcripts contain everything the agent saw: prompts, file contents, command output. Glassbox does not change that fact; it reads what is already on disk.

4. Files it writes

command	writes	contains	lifetime
<code>glassbox open <id></code>	<code><temp>/glassbox-<id8>.html</code> (<code>%TEMP%</code> on Windows, <code>\$TMPDIR</code> elsewhere)	the viewer with the transcript embedded	not deleted automatically; <code>glassbox clean</code> removes them. Use <code>--out PATH</code> to choose the location, <code>--no-open</code> to skip the browser
<code>glassbox open <id> --out F, compare A B --out F</code>	the file named	viewer + transcript(s)	until deleted
<code>glassbox check --legend F</code>	<code>F</code>	<code>salt + key</code> → path map for every file path in the report	until deleted; keep it local, it is the key to the redacted report
<code>glassbox fence</code>	<code>~/.claude/glassbox/fence.key</code> (created once, owner-only)	64 hex characters: the key its fingerprints are made with	until deleted; keep it on the machine — deleting it only means new fingerprints
<code>glassbox fence --shred</code>	the scanned files, in place (via <code><file>.glassbox-shred~</code> + rename)	the same files with each secret replaced by <code>[FENCED: <rule>:<fingerprint>]</code>	permanent, no backup
<code>glassbox hook install</code>	<code>~/.claude/settings.json</code> (+ <code>settings.json.glassbox-backup</code>)	a <code>Stop</code> hook entry, plus <code>SessionStart</code> with <code>--context</code> and <code>PreToolUse</code> with <code>--guard</code>	until <code>hook uninstall</code>
<code>Stop hook with --</code>	<code><temp>/glassbox-hook/<session-id>.json</code>	ids and titles of findings already handed back (counts	not deleted automatically

command	writes	contains	lifetime
feedback / plugin option feedback		and rule names; a title can name a tool or, for duplicate-subagent-read , a file path)	
Stop hook with -- context / plugin option context	<project>/ .glassbox/last-session.md and <project>/ .glassbox/.gitignore	the findings report for that session (≤ 6,000 chars; quotes tool inputs and outputs the way check --format md does) and, with feedback on, the agent's one-line reply	deleted by the hook after a clean session; .gitignore keeps it out of git
viewer Export report / Export redacted	a download the user saves	Markdown findings / structure-only .jsonl	user's choice

Nothing is written anywhere else. There is no config file, no cache and no state outside the rows above.

5. Processes it starts

- open and watch open the browser through the platform opener: cmd /c start "" <file> on Windows, open on macOS, xdg-open on Linux, or whatever GLASSBOX_BROWSER names. --no-open suppresses it.
- The Stop and SessionStart hooks run node once per event; the PreToolUse guard, if installed, runs node once before every tool call (see §6).
- Nothing else: no daemons, no scheduled tasks, no services, no shell-outs to anything but the opener above.

6. The hooks, exactly

Claude Code runs hooks as the user, with the hook's JSON on stdin and a JSON reply on stdout. The plugin registers two (Stop, SessionStart). glassbox hook install adds one to three: Stop always, SessionStart with --context , PreToolUse with --guard .

Stop (every time Claude Code finishes a turn). Reads the transcript named in the hook input, runs the rules, and returns a one-screen summary (turns, tool calls, context peak, cost, top findings) that Claude Code shows in the terminal. Timeout 60 s. Always exits 0 — it never blocks a session because it failed. With **feedback** on (off by default), when a finding at or above the threshold has not been shown before in that session, the hook returns decision: "block" once with the findings as the reason; Claude reads them, answers in a sentence what it would do differently, then stops. That extra turn costs tokens (one short reply) and happens at most once per new finding per session.

SessionStart (does something only with --context / plugin option context ; off by default — the CLI installs it only with --context , the plugin always registers it and it adds nothing to the session while the option is off). If <project>/ .glassbox/last-session.md exists, its text is returned as additionalContext , so the next session in that project starts knowing what went wrong last time. Timeout 15 s (plugin) or 60 s (CLI).

Two things to know about **context** before turning it on for a team:

1. It puts a file in the project directory whose contents are fed to the next session's model. The file is written by the hook from the transcript, and it is git-ignored, but anyone who can write to that

directory can change what the next session is told. Treat `.glassbox/` like any other file that shapes agent behaviour (`CLAUDE.md`, `.claude/settings.json`).

2. The notes quote from the session (tool inputs and outputs, up to 6,000 chars). On a shared project directory that is data at rest.

PreToolUse — the loop guard (CLI only, installed with `hook install --guard`; off by default; not in the plugin). Runs before every tool call, for every tool. It reads the last 4 MB of the session's transcript (for a subagent's call, the subagent's own transcript) and looks only at the current turn. If the call about to run is the same as one that has already failed twice in a row — same tool, same input, ignoring `description`, `timeout` and `run_in_background` — with nothing in between but reads, it returns `permissionDecision: "deny"` with a reason that quotes the first line of the last error (≤ 300 chars); Claude Code doesn't run that call and the model gets the reason instead. In every other case it prints nothing, which leaves Claude Code's normal permission flow in charge. It never returns `allow` and never changes a tool's input. A success, an edit, any other non-read call, a new human prompt, or the same call rejected or interrupted by a person resets the count. It writes nothing and opens no sockets; on any error it prints nothing and exits 0. Timeout 10 s. Replayed over 4,121 tool calls from 33 real sessions, it denied none.

Because it starts `node` before every tool call, `hook install --guard` writes a direct command, `node "<install dir>/bin/glassbox.mjs" hook ... --guard`, for every hook it installs, and refuses to run from an `npx` cache, where each call would cost seconds. It is not in the plugin: a plugin's hooks run for every user of the plugin, whether or not they want the guard.

How the CLI hook is invoked. `glassbox hook install` writes the command `npx -y glassbox-trace hook ...` into `settings.json`, so each run resolves the package through `npm` (cached after the first). To pin: `npm i -g glassbox-trace@0.9.1`, then `glassbox hook install --command glassbox ...`, or use the plugin, whose hooks run `node ${CLAUDE_PLUGIN_ROOT}/hooks/glassbox-hook.mjs` — the bundled copy, no `npx`. With `--guard` the command is the direct `node ".../bin/glassbox.mjs"` form above, so nothing goes through `npx`. If a CLI hook is present, the plugin's hooks see it and do nothing, so nothing runs twice.

Glassbox never approves or alters a tool call. The only PreToolUse hook is the opt-in guard above, and its only decision is to deny.

6b. Secrets in transcripts — `glassbox fence`

The transcripts Claude Code writes are plain text, never expire, and contain everything the agent read. If a developer's session opened a `.env`, ran `git remote -v` against a URL with a token in it, or pasted a key into the prompt, that value is now in a file under `~/claude/projects` — on the laptop, in its backups, and in any copy attached to a bug report. Nothing in the Claude Code toolchain looks back at those files.

`glassbox fence` does. It scans a session, a file, a folder, or every session under the home — and, in that last case, the prompt history (`history.jsonl`), the paste cache, file-history snapshots, debug logs and shell snapshots too (`--sessions-only` to skip them; binaries and files over 64 MB are skipped) — for: credentials whose format identifies them (AWS access key ids, GitHub / Anthropic / OpenAI / Slack / Stripe / Google / npm / SendGrid tokens, private key blocks, database URLs with a password) — **error**; secrets named by their context (`password=`, `api_key:`, `Bearer ...`, basic-auth URLs, JWTs) when the value has the entropy of a real one and is not a placeholder — **warn**; and reads of credential files (`.env`, `.npmrc`, `.netrc`, `~/aws/credentials`, `id_rsa`, `*.pem`, `secrets.json` ...) — **info**, because the file's contents are in the transcript whether or not a pattern matched them.

- **What the report holds.** A masked preview (`ghp_...r8` (40 chars)), a fingerprint, the rule, the line, and where it sits (user prompt, assistant text, tool input, tool result, with the tool and — for a Read — the path; or the store: prompt history, paste cache, file history, debug log, shell snapshot). Never the value. The fingerprint is the first 8 hex of **HMAC-SHA256 under a key kept on the**

machine (`~/ .claude/glassbox/fence.key`), so a report holding the fingerprint of a weak password gives nobody anything to test guesses against. It recognises the same key across sessions on one machine; to correlate across machines, give them one shared key with `--key FILE`, and keep that file away from the reports.

- `--shred` overwrites each value *in the file it was found in* with `[FENCED:<rule>:<fingerprint>]`. No backup is kept — a backup of a secret is the thing being removed. Every rewritten line is parsed again before anything is written; if one would not parse, or the file is not plain UTF-8, the file is left untouched and the refusal is reported. Files are written to a temporary sibling and renamed over the original; line endings are preserved; files with nothing to shred are not rewritten. A shredded transcript still opens in the viewer and still checks (no cost or diagnostic rule reads the content of a tool result).
- **Network:** none. Nothing is verified against a provider. Exit 1 when something at or above `--fail-on` (default `error`) was found, so it can run as a scheduled task or CI step: `glassbox fence --since 7d --format json --out fence.json`.
- **Limits.** A password in prose with no context word, or a token in a format the table does not know, passes through. Treat a clean sweep as "nothing the rules know about", not as proof.
- **The habit this should set:** rotate the key first (the row's advice line says where), then shred. Shredding without rotating removes the evidence, not the exposure.

6c. CLAUDE.md compliance — `glassbox adhere`

`glassbox adhere` reads the project's instruction files (`CLAUDE.md`, `.claude/CLAUDE.md`, `CLAUDE.local.md`, `~/ .claude/CLAUDE.md`) and the project's own transcripts, and reports per rule how often the agent obeyed it. It reads nothing else and writes nothing unless `--out` is given. Nine rule shapes are checkable mechanically (run X before commit, use A not B, never run / never touch, ask before, read before edit, commit message format, no new docs); the rest are listed as not checkable. Prompts, commands and paths appear in the evidence unless `--redact` is passed, in which case they become `«N chars»` and only the rule text (the team's own `CLAUDE.md`) remains. Network: none. See docs/ADHERE.md.

7. What leaves the machine when you share a report

Nothing leaves unless a person sends it. The formats built for sending are:

- `check --redact --format json|md` — every string from the transcript is dropped (prompts, commands, results, titles, paths); what remains is tool names, counts, timings, token figures and cost. Where a finding would quote something it says `«240 chars»`.
- `check --redact --legend F` — the same, except file paths become keys (`file:1a2b3c4d`, the first 8 hex of an HMAC-SHA256 with a random salt), so the report can say "this one file was read 48 times, 6 failed" without naming it. The key → path legend is written to `F` on the machine that ran the command and is not part of the report. `glassbox reveal report.md --legend F` reverses it, locally.
- The viewer's **Export redacted** — a structure-only `.jsonl` (timestamps, usage, tool names; no text).
- `glassbox collect <folder>` — the fleet report built from those redacted JSONs (one per machine). It contains nothing the inputs didn't: totals, rule ids and counts, source names (chosen by whoever named the file), session ids and keyed file names. A report that was not made with `--redact` is skipped, not merged, unless `--allow-unredacted` is passed.

Anything else (`open --out` , a plain `check` report, a transcript) contains session text and should be treated as such.

8. Cost figures

Costs are estimates from a rate card of Anthropic list prices (README, "Token and cost accounting"), or the `total_cost_usd` an Agent SDK run reports. Teams on negotiated rates, seats or credits pass their own card with `--rates FILE` or `GLASSBOX_RATES`; unknown models show "—" rather than a wrong number. The token counts are exact — they are read from the transcript.

9. Removal

- CLI hook: `glassbox hook uninstall` (removes every entry it added, the guard's `PreToolUse` entry included; `settings.json.glassbox-backup` is left for you to delete).
- Plugin: `/plugin uninstall glassbox@glassbox-trace`.
- Package: `npm uninstall -g glassbox-trace`; `npm cache clean --force` clears them).
- Files: `glassbox clean` removes `<temp>/glassbox-*.html` and `<temp>/glassbox-hook/`; any `.glassbox/` directory in a project, any legend, report or `--out` file you created are yours to delete.

Nothing else was changed on the machine.

10. Provenance and verification

- Source: github.com/aldohushi1-stack/glassbox, MIT. The engine is one file, `src/trace-core.js`, with no DOM and no imports; the CLI is `src/cli.mjs`, with `src/collect.mjs`, `src/tail.mjs`, `src/textfile.mjs`, `src/fence.mjs`, `src/adhere.mjs` and `src/guard.mjs`. It is small enough to read.
- npm releases are published by the repository's GitHub Actions workflow with `npm publish --provenance`, so npm shows which commit and workflow built each version.
- `npm test` runs the unit suite (parser, every rule, cost, redaction, legend, CLI, hooks and the guard, collect, fence, adhere, offline check); `npm run e2e` runs the browser suite with the request log; `npm run audit` runs the accessibility gate. All three run in CI on every push.
- Security reports: see [SECURITY.md](#) — privately, never in a public issue.
- To check a build yourself: open `dist/glassbox.html` with the browser's network panel open. It should show one entry, the file.

Questions: hi@aldo.ltd · Aldo Hushi, BlueprintAU, Adelaide.